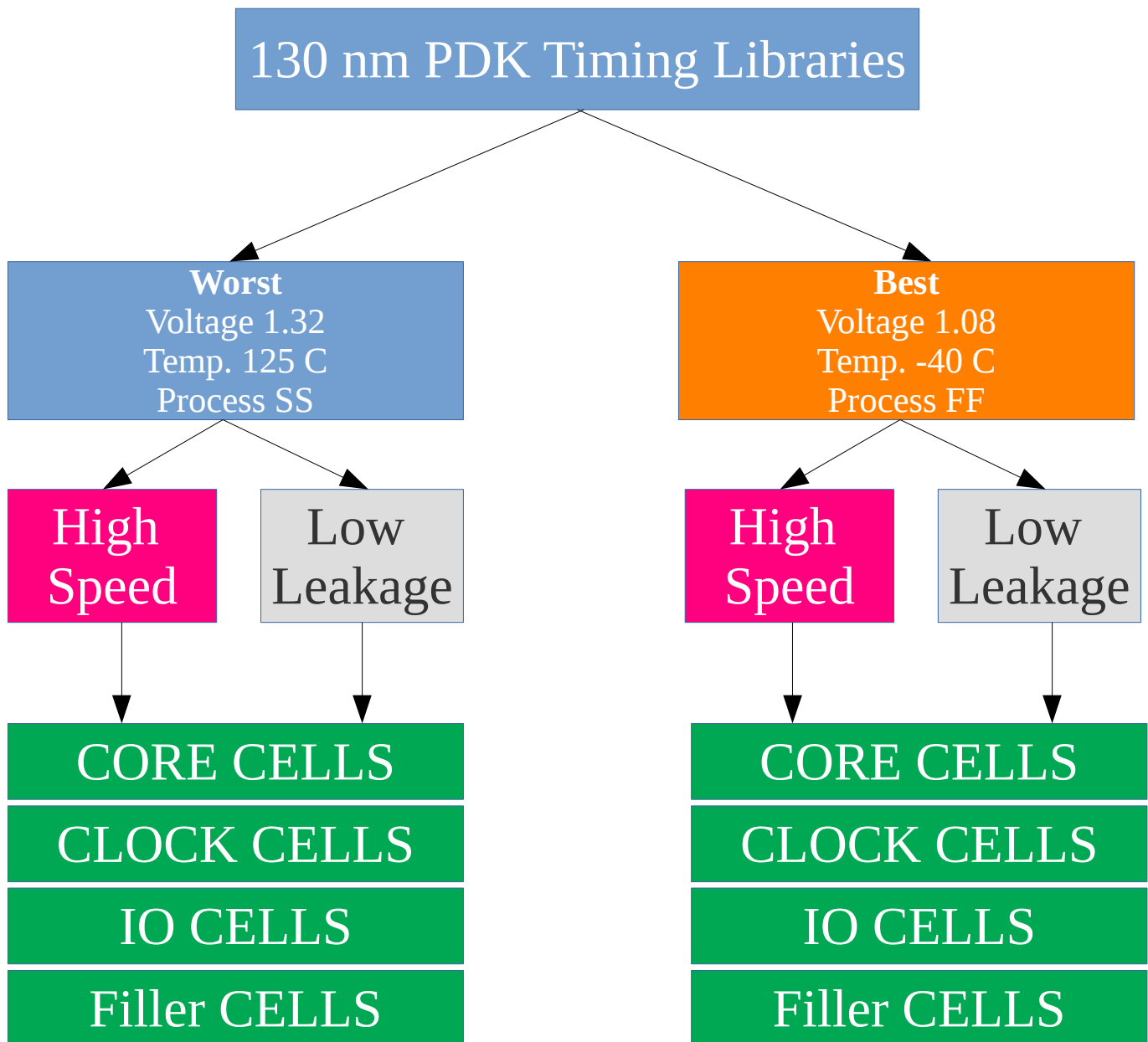


UE VLSI-2 TP: RTL Compiler

Matthieu Tuna

matthieu.tuna@gmail.com



For further details You can refer to (Process Design Kit) PDK documents

In this directory :

/users/soft/techno/dev/grenoble/hcmos9gp_920/DK_hcmos9gp_A_M6_2V5_HSLL_9.2/doc/Manuals

!!!!DON'T Copy PDK files to your Directory

!!!!DON'T Copy PDK files to your Directory

1. Set the environment

To be able to run Cadence tools, you must setup the environment. Add to your environment:

```
> export LM_LICENSE_FILE= 30000@cadence.cnfm.fr
> export PATH=$PATH:/users/soft/opus/Linux/RC-13.12-000/bin
> export PATH=$PATH:/users/soft/opus/Linux/INCISIV-15.20-034/tools/bin
> export PATH=$PATH:/users/soft/opus/Linux/EDI-13.24.026/bin
> export PATH=$PATH:/users/soft/opus/Linux/CONFORMAL-21.20-200/bin
```

Alternatively you can source this file in your terminal `~emad/mocca/TP2/cadence_tools.sh`

2. Load the libraries

The libraries are in this directory:

`/users/soft/techno/dev/grenoble/hcmos9gp_920/...`

There are two types available `_Best.lib` and `_Worst.lib`:

```
../CLOCK9GPHS_SNPS_AVT_4.1/SNPS/bc_1.32V_0C_wc_1.08V_125C/PHS/CLOCK9GPHS_Worst.lib
../CORE9GPHS_SNPS_AVT_4.1.a/SNPS/bc_1.32V_m40C_wc_1.08V_125C/PHS/CORE9GPHS_Worst.lib
../PR9M6_SNPS_AVT_2.1.a/SNPS/bc_1.32V_m40C_wc_1.08V_125C/PHS/PR9M6_Worst.lib
../IOLIB_65_M6_LL_HCMOS9GP_SNPS_AVT_7.1.a/SNPS/bc_1.32V_m40C_wc_1.08V_125C/PHS/
IOLIB_65_M6_LL_Worst.lib
../IOLIB_65_3V3_M6_LL_50A_HCMOS9GP_SNPS_AVT_7.1/SNPS/bc_1.32V_m40C_wc_1.08V_125C/PHS/
IOLIB_65_3V3_M6_LL_50A_Worst.lib

../CLOCK9GPHS_SNPS_AVT_4.1/SNPS/bc_1.32V_0C_wc_1.08V_125C/PHS/CLOCK9GPHS_Best.lib
../CORE9GPHS_SNPS_AVT_4.1.a/SNPS/bc_1.32V_m40C_wc_1.08V_125C/PHS/CORE9GPHS_Best.lib
../PR9M6_SNPS_AVT_2.1.a/SNPS/bc_1.32V_m40C_wc_1.08V_125C/PHS/PR9M6_Best.lib
../IOLIB_65_M6_LL_HCMOS9GP_SNPS_AVT_7.1.a/SNPS/bc_1.32V_m40C_wc_1.08V_125C/PHS/
IOLIB_65_M6_LL_Best.lib
../IOLIB_65_3V3_M6_LL_50A_HCMOS9GP_SNPS_AVT_7.1/SNPS/bc_1.32V_m40C_wc_1.08V_125C/PHS/
IOLIB_65_3V3_M6_LL_50A_Best.lib
```

Explain the difference between these two different types.

Which one is needed for the synthesis step ? Explain your choice.

Pick up a timing arc in the WC lib and the same timing arc in the BC, what is the value in the first library and in the second library. Add in your report a figure of the cell and the corresponding timing arc.

Open a ".tcl" file to store the RTLC commands. The command in RTLC to specify the library to be used is:

```
> set_attribute library library.lib
```

3. Load the design

The design is in the directory `~emad/mocca/TP2/SYNTH/rtl`

in RTLC you to load the design, use:

```
> set rtl [list \
>   ~emad/mocca/TP2/SYNTH/rtl/mips_32_1p_mul_div.vhd \
>   ]
>
> read_hdl -vhd1 $rtl
```

4. Elaborate

To elaborate the design enter the following command:

```
> elaborate MIPS_32_1P_MUL_DIV
```

The "elaborate" step does not synthesize the design, we will synthesize the design later on. What is the difference between elaboration and synthesis ?

You can now navigate throughout the design using the GUI and the unix-like database. To get the GUI opened, you have to add the option "-gui" when you launch rc: rc -gui .

5. Check design

GIGO: Garbage In-Garbage Out

If the loaded design is incorrect the generated netlist will also be incorrect. Just after elaboration you can check if everything is ok:

```
> check_design
```

Is everything OK ? Please explain what are the problems. The most important one is the 'Unresolved References' ?

6. Synthesis

```
> synthesize -to_mapped
```

Check the design to see if everything is fine.

7. Reset

This signal should be handled carefully. What kind of reset is used in this design ? Synchronous or asynchronous ? Please explain the differences between both and which one is used in the design. What is a reset synchronizer, please explain why a circuit should have a reset synchronizer ? Is there any reset synchronizer ? Code a reset synchronizer for asynchronous reset and add it to the design. Add in your report the figure and the waveforms of your reset synchronizer.

8. Reporting

The command:

```
> report
```

generates various reports such as: area, timing, datapath, clock_gating etc.

Report the area:

```
> report area
```

What is the area of the design ?

How many cells are used ?

Which type of wireload model is used ? Why ? What are the other types ?

What wireload model are used for each hierarchy ? Why ?

Let's have a look at the timing:

```
> report timing
```

This command print the timing on the longest path. Examine the path (start point / end point etc.) and explain it.

What is the timing of the path ?

What is the supposed max frequency the design is able to run ?

You can see at the end of the report the following comment:

```
> Timing slack : UNCONSTRAINED
```

What is a 'timing slack' ?

Why it is 'UNCONSTRAINED' ?

In order to check if the design is correctly constrained run:

```
> report timing -lint
```

This command categorized the errors in 3 types, explain those types.

8/ Constraints

Open a file .sdc where we will write the timing constraints.

a/ Reg-To-Reg

CLOCK

The port CK is the clock, the frequency of the clock should be 500MHz clock. Please write the corresponding SDC command.

Add in the RTLC script the command:

```
> read_sdc mips.sdc
```

Is the report timing -lint better ?

Redo the synthesis with the new constraints and report the timing.

Why does the report show the clock now ?

What is the worst path ?

What is the timing ?

Is the slack positive ?

Report the 10 worst paths.

What is a launch clock ?

What is a capture clock ?

What is your worst path ? Is the timing met ?

b/ Input-To-Reg

Let's consider that the inputs in the clock domain CLOCK takes 1.2 nanosecond to come. Even before running the tool can you already know if the timing will be met ? Explain (drawing + maths).

Please write the corresponding SDC command and report the timing. Is the result match your expectation ?

As it is an unrealistic constraint at this frequency, let's consider the minimum input constraint. Open the library .lib file and find the CK-> Q timing (access time). Takes this value as input delay. Rerun the tool and report the new timing. What does it mean ? Is the timing met ? What is the timing slack ?

c/ Reg-To-Output

First, let's consider an output delay on all outputs, synchronized to CLOCK. The output delay is 1.2 ns. Please write the correct SDC command and report the timing.

Is the timing met ? What is the timing slack ?

What is the worst path ? Why is it so long ?

Is the timing met now ?

9/ Report timing

Thanks to the report timing command you can check every single path you need to examine. The report timing command supports several option to specify a path: -from / -to / -through .

A/ Through

Print the timing path through the output Q of a flip-flop of your choice. Draw the path and explain it.

B/ From

Select a start point in the design and print the timing path starting from this point.

C/ To

Select a end point in the design and print the timing path ending at this point.

D/ From-To

Select a start point and a end point as well, print the timing path between these two points.

10/ Optimizations

By default the tool will optimize only the worst negative slack (WNS). WNS optimization produces better area but more violations. You can tell the synthesizer to optimize the total negative slack

(TNS). The TNS approach produces fewer violations, meaning fewer issues in the place-and-route stage, but on the other hand runtime will increase. As a general rule, always turn on TNS optimization.

```
> set_attribute endpoint_slack_opto true /
```

Explain the difference between WNS and TNS.

11/ Writing outputs

Write out the generated netlist and the sdc:

```
> write_hdl
```

```
> write_sdc
```

Open the generated netlist and check if everything seems ok. Open up the generated sdc and compare it to the input one. What are the differences ? What is a "set_dont_use" ? Why the tool should not use some cells ?

12/ Low Leakage Cells

So far we have used high speed cells (HS) so can you reload the low leakage (LL) cells and re-synthesis The design then report Time?Power ? Is there is a difference ?

13/ Equivalence checking (BONUS)

RTLc is able to generate the script to automatically run LEC (Conformal)

- what is an equivalence checking tool ?
- why do we need to run such a tool ?
- why do not use simulation ?

Run LEC and check that your netlist is equivalent to the original RTL.

14/ Design for Test: DfT(BOUNUS)

The insertion of the scan-chains are done during the synthesis. Explain what is a scan-chain and the purpose of it ?

Insert 4 scan-chains using fonctionnal pins for scan-ins and scan-outs.

For timing constraints, are you using the same sdc file as the functional mode ? Why not ?

Write the corresponding .sdc for the test mode as well as update the .sdc for the functional mode.